

DIGITAL PAYMENT SYSTEM

Java Mini Project Report

This report contains the complete Java source code, algorithm, UML class diagram, and a sample execution output for a console-based Digital Payment System.

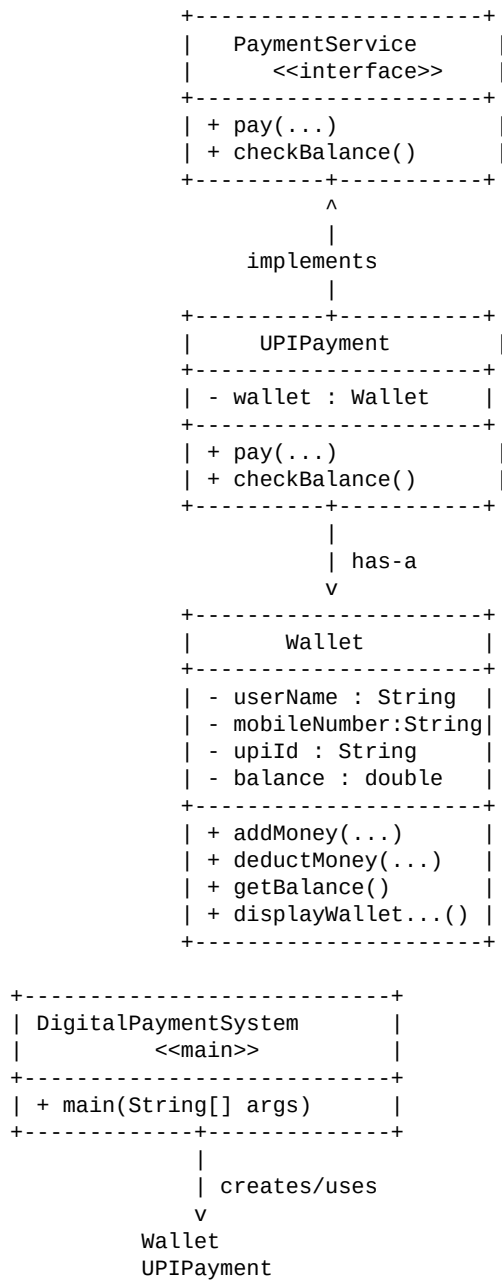
Feature	Description
Wallet	Creates and stores user, mobile, UPI ID and balance details.
Add Money	Adds a valid positive amount to the wallet.
UPI Payment	Validates receiver UPI ID, amount and available balance.
Balance	Displays the current wallet balance.
Exceptions	Uses custom exceptions for invalid UPI, invalid amount and insufficient balance.

1. Algorithm

1. Start the program.
2. Create Scanner object for user input.
3. Initialize wallet and payment objects as null.
4. Display the Digital Payment System menu.
5. Read the user's choice.
6. If choice is 1:
 - a. Check whether a wallet already exists.
 - b. Read name, mobile number and UPI ID.
 - c. Create a Wallet object.
 - d. Create a UPIPayment object.
7. If choice is 2:
 - a. Check whether the wallet exists.
 - b. Read the amount.
 - c. Reject zero or negative amounts.
 - d. Add the valid amount to the wallet balance.
8. If choice is 3:
 - a. Check whether the wallet exists.
 - b. Read receiver UPI ID and payment amount.
 - c. Validate the UPI ID.
 - d. Validate that the amount is greater than zero.
 - e. Check whether sufficient balance is available.
 - f. Deduct the payment amount if all checks pass.
9. If choice is 4, display the available balance.
10. If choice is 5, display wallet details.
11. If choice is 6, terminate the menu loop.
12. Handle custom exceptions and display appropriate error messages.
13. Display final wallet details if a wallet was created.
14. Close Scanner.
15. Stop.

2. UML Class Diagram

The diagram shows the interface, implementing class, Wallet class, main class, and custom exception hierarchy used by the program.



Exceptions:

InvalidUPIException

InvalidAmountException

InsufficientBalanceException

|
+---- all extend ----> Exception

3. Complete Java Code

```
import java.util.Scanner;

interface PaymentService {
    void pay(String upiId, double amount)
        throws InvalidUPIException, InvalidAmountException,
            InsufficientBalanceException;
    void checkBalance();
}

class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

class InvalidUPIException extends Exception {
    public InvalidUPIException(String message) {
        super(message);
    }
}

class InvalidAmountException extends Exception {
    public InvalidAmountException(String message) {
        super(message);
    }
}

class Wallet {
    private String userName;
    private String mobileNumber;
    private String upiId;
    private double balance;

    public Wallet(String userName, String mobileNumber, String upiId) {
        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upiId = upiId;
        this.balance = 0;
    }

    public void addMoney(double amount) throws InvalidAmountException {
        if (amount <= 0)
            throw new InvalidAmountException(
                "Amount must be greater than zero.");
        balance += amount;
        System.out.println("■" + amount + " added successfully.");
    }

    public void deductMoney(double amount) {
        balance -= amount;
    }

    public double getBalance() {
        return balance;
    }

    public void displayWalletDetails() {
        System.out.println("\n----- Wallet Details -----");
    }
}
```

```

        System.out.println("User Name      : " + userName);
        System.out.println("Mobile Number : " + mobileNumber);
        System.out.println("UPI ID       : " + upiId);
        System.out.println("Balance      : ■" + balance);
    }
}

class UPIPayment implements PaymentService {
    private Wallet wallet;

    public UPIPayment(Wallet wallet) {
        this.wallet = wallet;
    }

    @Override
    public void pay(String upiId, double amount)
        throws InvalidUPIException, InvalidAmountException,
            InsufficientBalanceException {

        if (upiId == null || !upiId.contains("@") ||
            upiId.startsWith("@") || upiId.endsWith("@") ||
            upiId.indexOf("@") != upiId.lastIndexOf("@"))
            throw new InvalidUPIException("Invalid UPI ID.");

        if (amount <= 0)
            throw new InvalidAmountException(
                "Payment amount must be greater than zero.");

        if (amount > wallet.getBalance())
            throw new InsufficientBalanceException(
                "Insufficient balance. Available balance: ■" +
                wallet.getBalance());

        wallet.deductMoney(amount);

        System.out.println("\n===== PAYMENT SUCCESS =====");
        System.out.println("Amount Paid : ■" + amount);
        System.out.println("Paid To      : " + upiId);
        System.out.println("Remaining    : ■" + wallet.getBalance());
        System.out.println("=====");
    }

    @Override
    public void checkBalance() {
        System.out.println(
            "Available Balance: ■" + wallet.getBalance());
    }
}

public class DigitalPaymentSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Wallet wallet = null;
        PaymentService payment = null;
        int choice;

        do {
            System.out.println("\n===== DIGITAL PAYMENT SYSTEM =====");

```

```

System.out.println("1. Create Wallet");
System.out.println("2. Add Money");
System.out.println("3. Make UPI Payment");
System.out.println("4. Check Balance");
System.out.println("5. Display Wallet Details");
System.out.println("6. Exit");
System.out.print("Enter your choice: ");
choice = sc.nextInt();
sc.nextLine();

try {
    switch (choice) {
        case 1:
            if (wallet != null) {
                System.out.println("Wallet already exists!");
                break;
            }

            System.out.print("Enter your name: ");
            String name = sc.nextLine();

            System.out.print("Enter mobile number: ");
            String mobile = sc.nextLine();

            System.out.print("Enter your UPI ID: ");
            String upi = sc.nextLine();

            wallet = new Wallet(name, mobile, upi);
            payment = new UPIPayment(wallet);
            System.out.println("Wallet created successfully!");
            break;

        case 2:
            if (wallet == null) {
                System.out.println(
                    "Please create a wallet first.");
                break;
            }

            System.out.print(
                "Enter amount to add: █");
            double addAmount = sc.nextDouble();
            wallet.addMoney(addAmount);
            System.out.println(
                "Current Balance: █" +
                wallet.getBalance());
            break;

        case 3:
            if (wallet == null) {
                System.out.println(
                    "Please create a wallet first.");
                break;
            }

            System.out.print(
                "Enter receiver UPI ID: ");
            String receiverUPI = sc.nextLine();

```

```

        System.out.print(
            "Enter payment amount: ■");
        double paymentAmount = sc.nextDouble();
        payment.pay(receiverUPI, paymentAmount);
        break;

    case 4:
        if (wallet == null) {
            System.out.println(
                "Please create a wallet first.");
            break;
        }
        payment.checkBalance();
        break;

    case 5:
        if (wallet == null) {
            System.out.println(
                "Please create a wallet first.");
            break;
        }
        wallet.displayWalletDetails();
        break;

    case 6:
        System.out.println(
            "Thank you for using Digital Payment System!");
        break;

    default:
        System.out.println(
            "Invalid choice. Please try again.");
    }
} catch (InvalidUPIException e) {
    System.out.println(
        "Transaction Failed: " + e.getMessage());
} catch (InvalidAmountException e) {
    System.out.println(
        "Transaction Failed: " + e.getMessage());
} catch (InsufficientBalanceException e) {
    System.out.println(
        "Transaction Failed: " + e.getMessage());
} finally {
    System.out.println(
        "Transaction processing completed.");
}
} while (choice != 6);

if (wallet != null) {
    System.out.println("\nFINAL WALLET DETAILS");
    wallet.displayWalletDetails();
}

sc.close();
}
}

```

4. Sample Output

===== DIGITAL PAYMENT SYSTEM =====

1. Create Wallet
2. Add Money
3. Make UPI Payment
4. Check Balance
5. Display Wallet Details
6. Exit

Enter your choice: 1

Enter your name: Rahul

Enter mobile number: 9876543210

Enter your UPI ID: rahul@upi

Wallet created successfully!

Transaction processing completed.

===== DIGITAL PAYMENT SYSTEM =====

1. Create Wallet
2. Add Money
3. Make UPI Payment
4. Check Balance
5. Display Wallet Details
6. Exit

Enter your choice: 2

Enter amount to add: ■5000

■5000.0 added successfully.

Current Balance: ■5000.0

Transaction processing completed.

===== DIGITAL PAYMENT SYSTEM =====

1. Create Wallet
2. Add Money
3. Make UPI Payment
4. Check Balance
5. Display Wallet Details
6. Exit

Enter your choice: 3

Enter receiver UPI ID: abc@upi

Enter payment amount: ■1500

===== PAYMENT SUCCESS =====

Amount Paid : ■1500.0

Paid To : abc@upi

Remaining : ■3500.0

=====

Transaction processing completed.

===== DIGITAL PAYMENT SYSTEM =====

1. Create Wallet
2. Add Money
3. Make UPI Payment
4. Check Balance
5. Display Wallet Details
6. Exit

Enter your choice: 4

Available Balance: ■3500.0

Transaction processing completed.

===== DIGITAL PAYMENT SYSTEM =====

1. Create Wallet
2. Add Money
3. Make UPI Payment
4. Check Balance
5. Display Wallet Details
6. Exit

Enter your choice: 5

----- Wallet Details -----

User Name : Rahul

Mobile Number : 9876543210

UPI ID : rahul@upi

Balance : ■3500.0

Transaction processing completed.

Enter your choice: 6

Thank you for using Digital Payment System!

Transaction processing completed.

FINAL WALLET DETAILS

----- Wallet Details -----

User Name : Rahul

Mobile Number : 9876543210

UPI ID : rahul@upi

Balance : ■3500.0

5. Concepts Used

Concept	Use in the Program
Interface	PaymentService defines payment operations.
Encapsulation	Wallet data members are private and accessed through methods.
Constructor	Wallet and UPIPayment objects are initialized through constructors.
Inheritance	Custom exception classes extend Exception.
Polymorphism	UPIPayment provides implementations of PaymentService methods.
Exception Handling	try-catch-finally handles transaction errors.
Scanner	Reads user input from the console.